

Chương 8: Máy RISC

Nội dung

- › Đặc điểm thực thi câu lệnh
- › Cấu trúc tập lệnh
- › Kỹ thuật ống dẫn trong RISC
- › RISC vs CISC

MỞ ĐẦU

- › Kiến trúc RISC là một đổi mới quan trọng trong lĩnh vực tổ chức máy tính.
- › Kiến trúc RISC là một nỗ lực để tăng thêm sức mạnh CPU bằng cách đơn giản hóa tập lệnh của CPU.
- › Xu hướng trái ngược với RISC là máy tính có tập lệnh phức tạp (CISC).
- › ***Cả kiến trúc RISC và CISC đã được được phát triển như một nỗ lực để che lấp khoảng cách ngữ nghĩa (Semantic gap).***

Semantic gap (1)

- › Hình thức phát triển rõ ràng nhất liên quan đến máy tính là (ngôn ngữ) lập trình.
 - Khi chi phí phần cứng giảm xuống, chi phí của phần mềm tăng lên.
 - Chi phí chính trong vòng đời của một hệ thống là phần mềm, không phải phần cứng.
 - Thông thường các chương trình, hệ thống và ứng dụng, sẽ tiếp tục xuất hiện các lỗi mới sau nhiều năm hoạt động.

Semantic gap (2)

- › Cần phát triển các ngôn ngữ lập trình bậc cao (High-level language, HLL) mạnh mẽ và phức tạp hơn là yêu cầu cấp thiết:
 - Nhiều ngôn ngữ lập trình đã được phát triển (Ada, C++, Java,...): mức độ trừu tượng cao, có sự đồng nhất, và mạnh mẽ.
 - Cho phép lập trình viên thể hiện thuật toán chính xác hơn.
 - Hỗ trợ một cách tự nhiên việc sử dụng lập trình có cấu trúc và / hoặc thiết kế hướng đối tượng.
 - *Điều này đã dẫn đến một vấn đề: sự khác biệt ngữ nghĩa (semantic gap).*

Semantic gap (3)

› Các cách tiếp cận:

- CISC: thiết kế rất phức tạp, bao gồm một số lượng lớn các lệnh và chế độ địa chỉ;
- RISC: đơn giản hóa tập lệnh và điều chỉnh nó cho đúng yêu cầu thực tế của chương trình

Đánh giá thực hiện chương trình

- › **Operations performed:** xác định các chức năng được thực hiện bởi bộ xử lý và tương tác của nó với bộ nhớ (tần suất thực thi các phép tính toán).
- › **Operands used:** Các loại toán hạng và tần suất sử dụng của chúng xác định tổ chức bộ nhớ để lưu trữ và các chế độ địa chỉ để truy cập chúng.
- › **Execution sequencing:** tần suất các lệnh nhảy, vòng lặp, gọi chương trình con

Operations performed

- › Tỷ lệ tần xuất thực thi lệnh máy
 - Các lệnh gán: 33%
 - Rẽ nhánh có điều kiện: 20%
 - Số học / logic: 16%
 - Các lệnh khác Khác: Từ 0,1% đến 10%
- › Chế độ đánh địa chỉ: phần lớn các lệnh sử dụng các chế độ địa chỉ đơn giản và chỉ được sử dụng bởi ~ 18% các lệnh.

Operands used

- › Tỷ lệ từ 74 đến 80% toán hạng là vô hướng (số nguyên, số thực, ký tự...) có thể được lưu trữ trong thanh ghi.
- › Phần còn lại (20-26%) là mảng / cấu trúc; 90% chúng là các biến toàn cục.
- › Có 80% số vô hướng là các biến cục bộ.
- › Phần lớn các toán hạng là các biến cục bộ của loại vô hướng, có thể được lưu trữ trong thanh ghi.

Lệnh CALL/RETURN các thủ tục

- › Chỉ 1,25% thủ tục được gọi là có hơn sáu tham số.
- › Chỉ 6,7% thủ tục được gọi có nhiều hơn sáu biến cục bộ.
- › Chuỗi các cuộc gọi thủ tục lồng nhau thường là ngắn và chỉ rất hiếm khi dài hơn 6 tham số.

Lệnh CALL/RETURN các thủ tục

- › Tỷ lệ phần trăm của tổng thời gian thực hiện dành cho việc thực hiện lệnh HLL.
 - Phần lớn thời gian được dành để thực hiện CALL/RETURN
 - Khi chỉ có 15% các lệnh HLL được thực hiện là CALL/RETURN, chúng vẫn được thực hiện hầu hết thời gian, vì độ phức tạp của chúng.
 - CALL/RETURN được biên dịch thành tương đối chuỗi dài các lệnh máy cần nhiều tham chiếu đến bộ nhớ.

Đánh giá thực thi chương trình

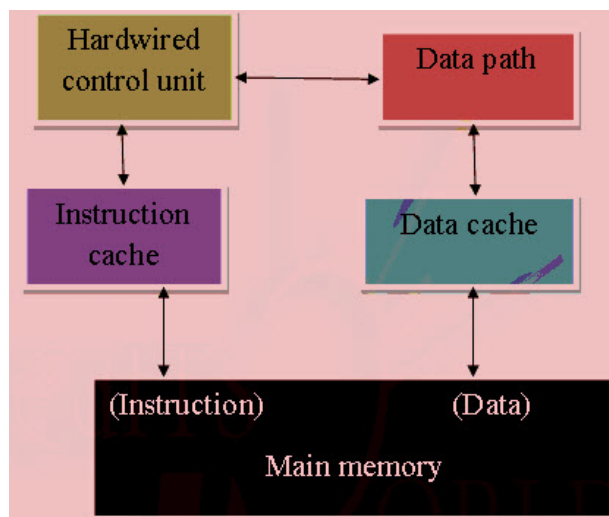
- › Các phép tính toán đơn giản (ALU và gán) phổ biến hơn là các phép phức tạp
- › Tần số truy cập toán hạng lớn; trung bình mỗi lệnh liên quan đến 1.9 toán hạng.
- › Hầu hết các toán hạng được tham chiếu là vô hướng (vì vậy chúng có thể được lưu trữ trong một thanh ghi) và chúng là các biến hoặc tham số cục bộ .
- › Tối ưu hóa các thủ tục CALL/RETURN sẽ tăng tốc độ thực thi chương trình.

- Những kết luận này là điểm khởi đầu của hướng tiếp cận: RISC

RISC

› RISC (viết tắt của Reduced Instructions Set Computer)

- Máy tính với tập lệnh đơn giản hóa: một phương pháp thiết kế các bộ vi xử lý (VXL) theo hướng đơn giản hóa tập lệnh, trong đó thời gian thực thi tất cả các lệnh đều như nhau.



Các hệ thống RISC phổ biến

- › ARM – palm, Inc.
 - Ban đầu sử dụng (CISC) Motorola 680x0 trong những PDA đầu tiên, nhưng hiện tại là (RISC) ARM.
 - Nhiều nhà sản xuất điện thoại di động, cũng dựa trên kiến trúc ARM.
- › MIPS, trong các máy tính SGI, PlayStation và Nintendo 64 game consoles.
- › POWER PC trong các SuperComputers/mainframes của IBM.
- › SPARC và UltraSPARC, trong tất cả các hệ thống của Sun.
- › ...

Các hệ thống RISC phổ biến

› So sánh một số hệ thống RISC và non-RISC.

	Complex Instruction Set (CISC) Computer			Reduced Instruction Set (RISC) Computer		Superscalar		
Characteristic	IBM 370/168	VAX 11/780	Intel 80486	SPARC	MIPS R4000	PowerPC	Ultra SPARC	MIPS R10000
Year developed	1973	1978	1989	1987	1991	1993	1996	1996
Number of instructions	208	303	235	69	94	225		
Instruction size (bytes)	2–6	2–57	1–11	4	4	4	4	4
Addressing modes	4	22	11	1	1	2	1	1
Number of general-purpose registers	16	16	8	40 - 520	32	32	40 - 520	32
Control memory size (Kbits)	420	480	246	—	—	—	—	—
Cache size (KBytes)	64	64	8	32	128	16-32	32	64

Đặc điểm chính thực thi lệnh trong RISC

- › Tập lệnh đơn giản và mỗi lệnh tương ứng một chu kỳ
- › Hầu hết các lệnh thực thi theo cơ chế **register to register**
- › Chế độ địa chỉ đơn giản
- › Định dạng lệnh đơn giản

Đặc điểm chính thực thi lệnh trong RISC

› *Mỗi lệnh tương ứng một chu kỳ (1)*

- Hầu hết các các lệnh trong RISC được thực thi trong một chu kỳ máy (sau khi được nạp và giải mã).
- Tập có các lệnh đơn giản nên giảm độ phức tạp của đơn vị điều khiển và đường dẫn dữ liệu; do đó, bộ xử lý có thể làm việc ở tần số đồng hồ cao.

Đặc điểm chính thực thi lệnh trong RISC

› *Mỗi lệnh tương ứng một chu kỳ (2)*

- Pipelines được sử dụng hiệu quả vì các lệnh là đơn giản và thời gian thực hiện tương tự.
- Các tính toán phức tạp trên RISC được thực thi như một chuỗi các lệnh đơn giản. Các lệnh máy có thể được gắn cứng (hardwired) tức là một chức năng xử lý được gắn vào các mạch điện tử của máy tính thay vì được khiển bởi các lệnh của chương trình

Đặc điểm chính thực thi lệnh trong RISC

› *Cơ chế thực thi register to register (1)*

- Mỗi chu kỳ máy được định nghĩa là thời gian cần thiết để tìm nạp hai toán hạng từ các thanh ghi, thực hiện thao tác tính toán ALU và lưu kết quả vào một thanh ghi.
- Chỉ có một vài lệnh truy cập bộ nhớ cần nhiều hơn một chu kỳ để thực hiện (sau khi được nạp và giải mã).

Đặc điểm chính thực thi lệnh trong RISC

› *Cơ chế thực thi register to register (2)*

- Chỉ các lệnh nạp (LOAD) và lưu trữ (STORE) tham chiếu dữ liệu trong bộ nhớ; tất cả các lệnh khác thực thi chỉ với các thanh ghi (theo cơ chế **register-to-register**).
- Một số lượng lớn các thanh ghi có sẵn: các biến và kết quả trung gian có thể được lưu trữ trong các thanh ghi và không yêu cầu lặp lại việc nạp và lưu trữ từ/đến bộ nhớ.

Đặc điểm chính thực thi lệnh trong RISC

› *Chế độ địa chỉ đơn giản*

- Hầu như tất cả các lệnh trong RISC sử dụng địa chỉ thanh ghi đơn giản.
- Các chế độ khác, phức tạp hơn có thể được tổng hợp trong phần mềm từ những chế độ đơn giản (tính năng thiết kế này góp phần đơn giản hóa tập lệnh và bộ điều khiển).

Đặc điểm chính thực thi lệnh trong RISC

› **Định dạng lệnh đơn giản**

- Các lệnh có độ dài cố định và định dạng thống nhất (chỉ có một hoặc một vài định dạng được sử dụng)
- Điều này làm cho việc nạp và giải mã lệnh đơn giản và nhanh chóng; không cần thiết để chờ cho đến khi độ dài của một lệnh được xác định để bắt đầu giải mã lệnh kế tiếp;
- Giải mã được đơn giản hóa vì mã thực thi (opcode) và trường địa chỉ được đặt ở cùng một vị trí cho tất cả các lệnh

Tập lệnh MIPS R4000

- › Một trong những bộ chip RISC thương mại đầu tiên được phát triển bởi MIPS Technology Inc.
- › Hệ thống được lấy cảm hứng từ một hệ thống thử nghiệm, cũng sử dụng tên MIPS, được phát triển tại Stanford.
- › R4000 sử dụng 64bit chứ không phải 32 bit cho tất cả các đường dẫn dữ liệu bên trong và bên ngoài cho địa chỉ, thanh ghi và ALU.

Tập lệnh MIPS R4000

- › Việc sử dụng 64 bit có một số lợi thế so với kiến trúc 32 bit.
 - Cho phép một không gian địa chỉ lớn hơn, đủ lớn để một hệ điều hành ánh xạ nhiều hơn **một terabyte** tệp trực tiếp vào bộ nhớ ảo để dễ dàng truy cập.
 - Với các ổ đĩa **1 terabyte** và lớn hơn hiện nay, không gian địa chỉ **4 gigabyte** của máy 32 bit trở nên hạn chế.
 - Dung lượng 64 bit cho phép R4000 xử lý dữ liệu, chẳng hạn như các số dấu phẩy động và chuỗi ký tự có độ chính xác kép, lên đến tám ký tự trong một xử lý.

Tập lệnh MIPS R4000

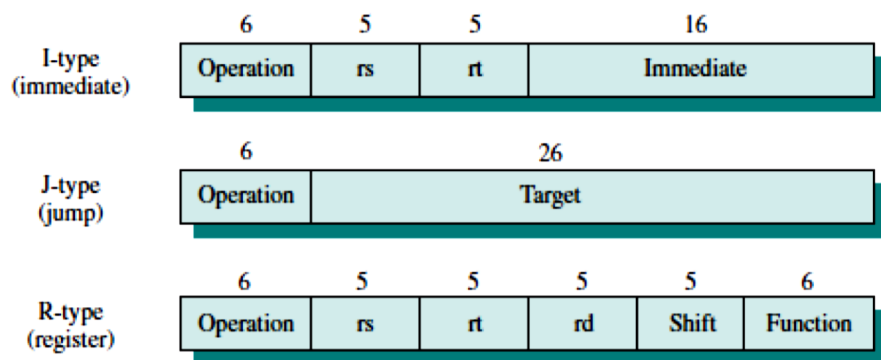
- › Chip xử lý R4000 được phân vùng thành hai phần,
 - Một phần chứa CPU và phần còn lại chứa bộ đồng xử lý để quản lý bộ nhớ.
 - Bộ xử lý có kiến trúc rất đơn giản. Mục đích là để thiết kế một hệ thống trong đó thực thi các lệnh logic và đơn giản nhất có thể.
 - Bộ xử lý hỗ trợ ba mươi hai thanh ghi 64 bit. C
 - Cung cấp tới 128 Kbyte bộ nhớ cache tốc độ cao, một nửa cho các lệnh và dữ liệu. Bộ đệm tương đối lớn cho phép hệ thống giữ các bộ mã chương trình và dữ liệu cục bộ lớn cho bộ xử lý, giảm tải cho bộ nhớ chính.

Tập lệnh MIPS R4000

- › MIPS R4000 không sử dụng mã điều kiện.
 - Nếu một lệnh khởi tạo một điều kiện, các cờ tương ứng được lưu trữ trong một thanh ghi mục đích chung. Điều này tránh sự cần thiết phải có cổng logic đặc biệt để xử lý các mã điều kiện vì chúng ảnh hưởng đến cơ chế đường ống và sắp xếp lại các lệnh của trình biên dịch.
 - MIPS sử dụng độ dài lệnh 32 bit duy nhất. Độ dài lệnh đơn này đơn giản hóa nạp và giải mã lệnh, và nó cũng đơn giản hóa sự tương tác của nạp lệnh với đơn vị quản lý bộ nhớ ảo (nghĩa là, các lệnh không vượt qua ranh giới từ hoặc trang nhớ).

Tập lệnh MIPS R4000

- › Ba định dạng lệnh dùng chung cho mã lệnh và tham chiếu thanh ghi, đơn giản hóa việc giải mã lệnh.



Operation	Operation code
rs	Source register specifier
rt	Source/destination register specifier
Immediate	Immediate, branch, or address displacement
Target	Jump target address
rd	Destination register specifier
Shift	Shift amount
Function	ALU/shift function specifier

Tập lệnh MIPS R4000

- › Chỉ có chế độ địa chỉ bộ nhớ đơn giản nhất và được sử dụng thường xuyên nhất được triển khai trong phần cứng.
 - Tất cả các tham chiếu bộ nhớ bao hàm phần bù 16 bit từ thanh ghi 32 bit.
 - Ví dụ: `lw r2, 128(r3)`
 - / * load word at address 128 offset from register 3 into register 2 */
 - Mỗi thanh ghi trong số 32 thanh ghi mục đích chung có thể được sử dụng làm thanh ghi cơ sở.

Bộ lệnh MIPS R-Series

Load/Store Instructions	
LB	Load Byte
LBU	Load Byte Unsigned
LH	Load Halfword
LHU	Load Halfword Unsigned
LW	Load Word
LWL	Load Word Left
LWR	Load Word Right
SB	Store Byte
SH	Store Halfword
SW	Store Word
SWL	Store Word Left
SWR	Store Word Right
Arithmetic Instructions (ALU Immediate)	
ADDI	Add Immediate
ADDIU	Add Immediate Unsigned
SLTI	Set on Less Than Immediate
SLTIU	Set on Less Than Immediate Unsigned
ANDI	AND Immediate
ORI	OR Immediate
XORI	Exclusive-OR Immediate
LUI	Load Upper Immediate

Arithmetic Instructions (3-operand, R-type)	
ADD	Add
ADDU	Add Unsigned
SUB	Subtract
SUBU	Subtract Unsigned
SLT	Set on Less Than
SLTU	Set on Less Than Unsigned
AND	AND
OR	OR
XOR	Exclusive-OR
NOR	NOR
Shift Instructions	
SLL	Shift Left Logical
SRL	Shift Right Logical
SRA	Shift Right Arithmetic
SLLV	Shift Left Logical Variable
SRLV	Shift Right Logical Variable
SRAV	Shift Right Arithmetic Variable
Multiply/Divide Instructions	
MULT	Multiply
MULTU	Multiply Unsigned
DIV	Divide
DIVU	Divide Unsigned
MFHI	Move From HI
MTHI	Move To HI
MFLO	Move From LO
MTLO	Move To LO

Bộ lệnh MIPS R-Series

Jump and Branch Instructions

J	Jump
JAL	Jump and Link
JR	Jump to Register
JALR	Jump and Link Register
BEQ	Branch on Equal
BNE	Branch on Not Equal
BLEZ	Branch on Less Than or Equal to Zero
BGTZ	Branch on Greater Than Zero
BLTZ	Branch on Less Than Zero
BGEZ	Branch on Greater Than or Equal to Zero
BLTZAL	Branch on Less Than Zero And Link
BGEZAL	Branch on Greater Than or Equal to Zero And Link

Coprocessor Instructions

LWCz	Load Word to Coprocessor
SWCz	Store Word to Coprocessor
MTCz	Move To Coprocessor
MFCz	Move From Coprocessor
CTCz	Move Control To Coprocessor
CFCz	Move Control From Coprocessor
COPz	Coprocessor Operation
BCzT	Branch on Coprocessor z True
BCzF	Branch on Coprocessor z False

Special Instructions

SYSCALL	System Call
BREAK	Break

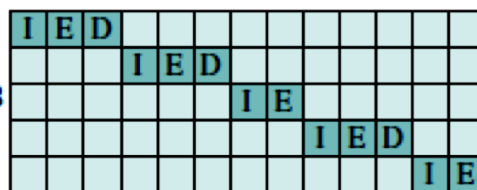
RISC Pipelining

- › Hầu hết các lệnh tuân theo cơ chế register to register và một kỳ lệnh có hai giai đoạn sau:
 - **I**: Nạp lệnh.
 - **E**: Thực thi, thực hiện thao tác ALU với thanh ghi nhập xuất.

- › Đối với hoạt động LOAD / STORE , ba giai đoạn được yêu cầu:
 - **I**: Nạp lệnh.
 - **E**: Thực thi. Xác định địa chỉ nhớ.
 - **D**: Bộ nhớ. Hoạt động theo cơ chế Register-to-memory hoặc memory-to-register.

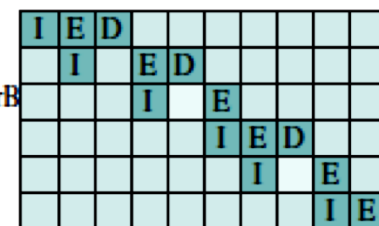
RISC Pipelining

Load $rA \leftarrow M$
 Load $rB \leftarrow M$
 Add $rC \leftarrow rA + rB$
 Store $M \leftarrow rC$
 Branch X



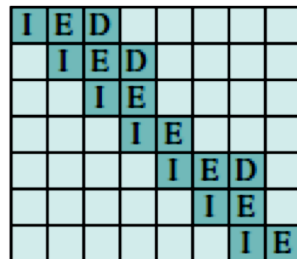
(a) Sequential execution

Load $rA \leftarrow M$
 Load $rB \leftarrow M$
 Add $rC \leftarrow rA + rB$
 Store $M \leftarrow rC$
 Branch X
 NOOP



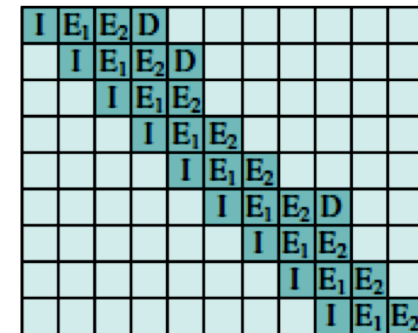
(b) Two-stage pipelined timing

Load $rA \leftarrow M$
 Load $rB \leftarrow M$
 NOOP
 Add $rC \leftarrow rA + rB$
 Store $M \leftarrow rC$
 Branch X
 NOOP



(c) Three-stage pipelined timing

Load $rA \leftarrow M$
 Load $rB \leftarrow M$
 NOOP
 NOOP
 Add $rC \leftarrow rA + rB$
 Store $M \leftarrow rC$
 Branch X
 NOOP
 NOOP



(d) Four-stage pipelined timing

RISC Pipelining

- › Một chuỗi các lệnh không sử dụng kỹ thuật ống dẫn: một quá trình lãng phí.
- › Một kỹ thuật ống dẫn rất đơn giản cũng có thể cải thiện đáng kể hiệu suất.

Load $rA \leftarrow M$
Load $rB \leftarrow M$
Add $rC \leftarrow rA + rB$
Store $M \leftarrow rC$
Branch X

I	E	D										
			I	E	D							
						I	E					
								I	E	D		
											I	E

(a) Sequential execution

RISC Pipelining

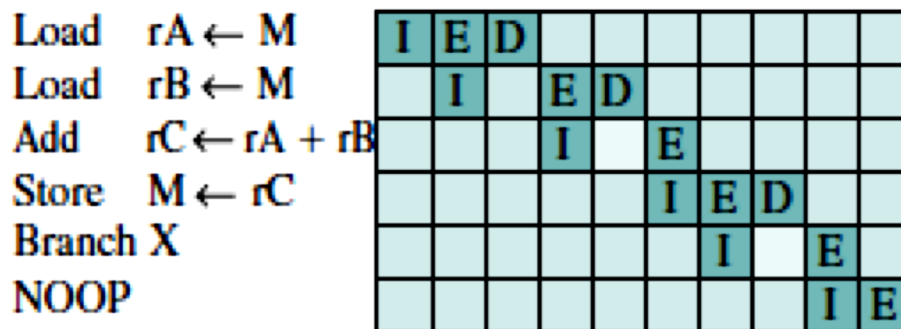
- › Kỹ thuật ống dẫn hai giai đoạn, trong đó giai đoạn **I** và **E** của lệnh khác nhau được thực hiện đồng thời: giai đoạn nạp lệnh và giai đoạn thực thi / bộ nhớ thực thi lệnh, bao gồm các thực thi **register-to-memory** / **memory-to-register**.

Load $rA \leftarrow M$	I	E	D							
Load $rB \leftarrow M$		I		E	D					
Add $rC \leftarrow rA + rB$				I		E				
Store $M \leftarrow rC$						I	E	D		
Branch X							I		E	
NOOP									I	E

(b) Two-stage pipelined timing

RISC Pipelining

- › Giai đoạn tìm nạp lệnh của lệnh thứ hai có thể được thực hiện song song với phần đầu tiên của giai đoạn thực thi / bộ nhớ.
- › Tuy nhiên, giai đoạn thực thi / bộ nhớ của lệnh thứ hai phải bị trì hoãn cho đến khi lệnh thứ nhất xóa giai đoạn thứ hai của đường ống.



(b) Two-stage pipelined timing

RISC Pipelining

- › Kỹ thuật ống dẫn hai giai đoạn có thể mang lại gấp đôi tốc độ thực hiện của sơ đồ nối tiếp.
- › Hai vấn đề ngăn cản việc tăng tốc tối đa đạt được.
 - Giả định rằng một bộ nhớ đơn được sử dụng và chỉ có một truy cập bộ nhớ cho mỗi giai đoạn. Điều này đòi hỏi phải chèn trạng thái chờ trong một số lệnh.
 - Một lệnh rẽ nhánh làm gián đoạn luồng thực thi tuần tự. Một lệnh NOOP có thể được chèn vào luồng lệnh bởi trình biên dịch.

RISC Pipelining

- › Hiệu ứng đường ống có thể được cải thiện hơn nữa bằng cách cho phép hai truy cập bộ nhớ cho mỗi giai đoạn.
 - Tối đa ba lệnh được chồng chéo và sự cải thiện nhiều đến **3 lần**
 - Các lệnh rẽ nhánh làm cho việc tăng tốc không đạt được đến mức tối đa có thể
 - Nếu một lệnh cần một toán hạng bị thay đổi bởi lệnh trước đó, việc trì hoãn là cần thiết và có thể được thực hiện bởi một NOOP.

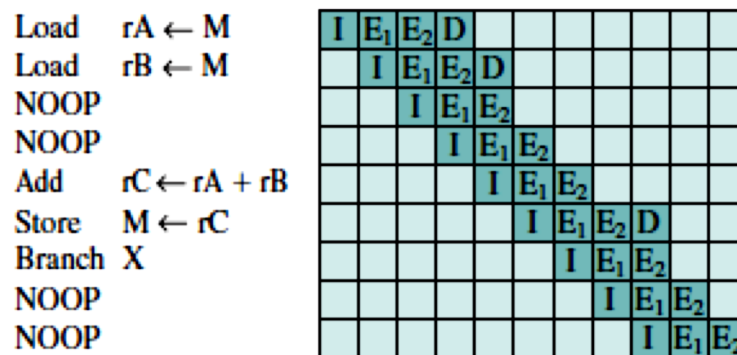
Load	$rA \leftarrow M$	I	E	D					
Load	$rB \leftarrow M$		I	E	D				
NOOP				I	E				
Add	$rC \leftarrow rA + rB$				I	E			
Store	$M \leftarrow rC$					I	E	D	
Branch	X						I	E	
NOOP								I	E

RISC Pipelining

- › Kỹ thuật ống dẫn hoạt động tốt nhất nếu ba giai đoạn có thời lượng xấp xỉ bằng nhau
- › Giai đoạn **E** thường liên quan đến hoạt động ALU, do vậy nó có thể dài hơn: có thể chia thành hai giai đoạn con:
 - **E1**: đọc tệp thanh ghi (**Register file read**)
 - **E2**: Thực thi tính toán ALU và ghi thanh ghi (**ALU operation and register write**)

RISC Pipelining

- › Do tính đơn giản và đều đặn của tập lệnh RISC, việc thiết kế (phân chia) giai đoạn thành ba hoặc bốn giai đoạn có thể dễ dàng thực hiện.
- › Một đường ống dẫn bốn giai đoạn: Có thể thực hiện tối đa bốn lệnh tại một thời điểm và khả năng tăng tốc tối đa đến **4 lần**.
- › Sử dụng NOOP để tính toán dữ liệu và độ trễ của rẽ nhánh.



(d) Four-stage pipelined timing

Optimization of Pipelining

- › Do tính chất đơn giản và đều đặn của các lệnh trong RISC, nên dễ dàng cho một thiết kế phần cứng để thực hiện một hiệu ứng đường ống đơn giản, nhanh chóng.
- › Tuy nhiên, dữ liệu và các lệnh rẽ nhánh làm giảm tổng thể tỷ lệ thực thi. Để giải quyết, các kỹ thuật sắp xếp lại mã đã được phát triển:
 - ***DELAYED BRANCH***
 - ***DELAYED LOAD***
 - ***LOOP UNROLLING***

Optimization of Pipelining

› ***DELAYED BRANCH***

- Một cách để tăng tính hiệu quả của kỹ thuật đường ống, bằng cách sử dụng một rẽ nhánh không có tác động (ảnh hưởng), cho đến sau khi thực hiện lệnh tiếp theo (do đó thuật ngữ bị trì hoãn).
- Vị trí lệnh ngay sau nhánh được gọi là khe trễ (*delay slot*).

Optimization of Pipelining

› **DELAYED BRANCH**

- Trong **normal branch**: các lệnh trong chương trình ngôn ngữ máy thông thường.
- Lệnh tại địa chỉ **102** được thực thi, lệnh tiếp theo được thực hiện là **105**. Để chuẩn hóa, một NOOP được chèn sau nhánh này (**delayed branch**). Hiệu suất tăng đạt được nếu các lệnh ở **101** và **102** được hoán đổi cho nhau (**optimized delayed branch**).

Address	Normal Branch		Delayed Branch		Optimized Delayed Branch	
100	LOAD	X, rA	LOAD	X, rA	LOAD	X, rA
101	ADD	1, rA	ADD	1, rA	JUMP	105
102	JUMP	105	JUMP	106	ADD	1, rA
103	ADD	rA, rB	NOOP		ADD	rA, rB
104	SUB	rC, rB	ADD	rA, rB	SUB	rC, rB
105	STORE	rA, Z	SUB	rC, rB	STORE	rA, Z
106			STORE	rA, Z		

Optimization of Pipelining

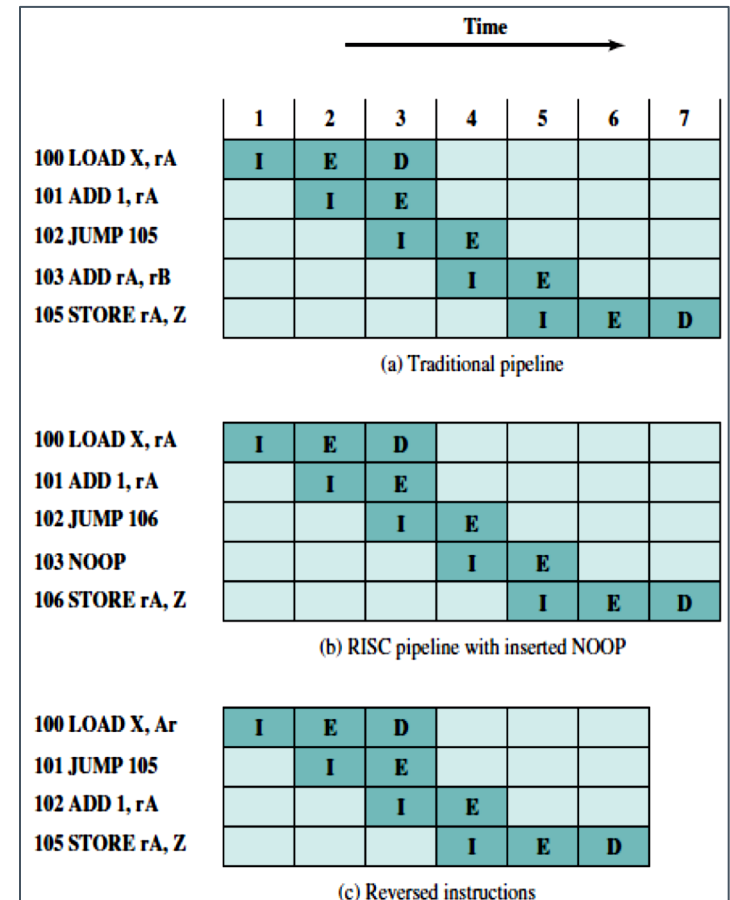
› ***DELAYED BRANCH***

a) Cách tiếp cận truyền thống đối với đường ống

b) Một đường ống được xử lý bởi một RISC điển hình. Thời gian xử lý là như nhau (so với a).

Do chèn lệnh NOOP nên không cần mạch đặc biệt để xóa đường ống; NOOP chỉ đơn giản là thực thi mà không có tác động gì.

c) sử dụng delayed branch. Lệnh JUMP được tìm nạp trước lệnh ADD. Tuy nhiên, lưu ý rằng lệnh ADD được nạp trước khi thực thi lệnh JUMP, do vậy có sự thay đổi bộ đếm chương trình. Lệnh ADD được thực thi cùng lúc với nạp lệnh STORE.



Optimization of Pipelining

› ***DELAYED LOAD***

- Các lệnh LOAD (tương tự STORE) yêu cầu truy cập bộ nhớ và việc thực thi chúng có thể được hoàn thành trong một chu kỳ đồng hồ duy nhất.
- Tuy nhiên, trong chu kỳ tiếp theo, một lệnh mới được khởi động bởi bộ xử lý.

Optimization of Pipelining

› **DELAYED LOAD**

– Hai giải pháp khả thi:

1. Phần cứng nên trì hoãn việc thực thi lệnh theo sau lệnh LOAD, nếu lệnh này cần nạp giá trị.
2. Một giải pháp hiệu quả hơn, dựa trên trình biên dịch, có điểm tương đồng với sự trì hoãn phân nhánh (delayed branch), là nạp chậm (**delayed load**) :
 - › Bộ xử lý luôn thực hiện lệnh theo LOAD, không có trì hoãn (lệnh này không cần giá trị được tải)

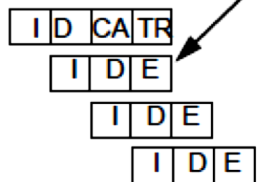
Optimization of Pipelining

› *DELAYED LOAD*

LOAD	R1,X	loads from address X into R1
ADD	R2,R1	$R2 \leftarrow R2 + R1$
ADD	R4,R3	$R4 \leftarrow R4 + R3$
SUB	R2,R4	$R2 \leftarrow R2 - R4$

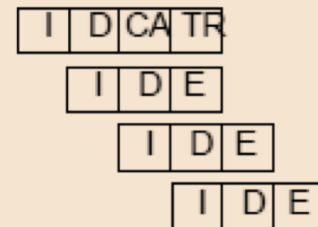
LOAD	R1,X	loads from address X into R1
ADD	R4,R3	$R4 \leftarrow R4 + R3$
ADD	R2,R1	$R2 \leftarrow R2 + R1$
SUB	R2,R4	$R2 \leftarrow R2 - R4$

LOAD R1,X
ADD R2,R1
ADD R4,R3
SUB R2,R4



a)

LOAD R1,X
ADD R4,R3
ADD R2,R1
SUB R2,R4



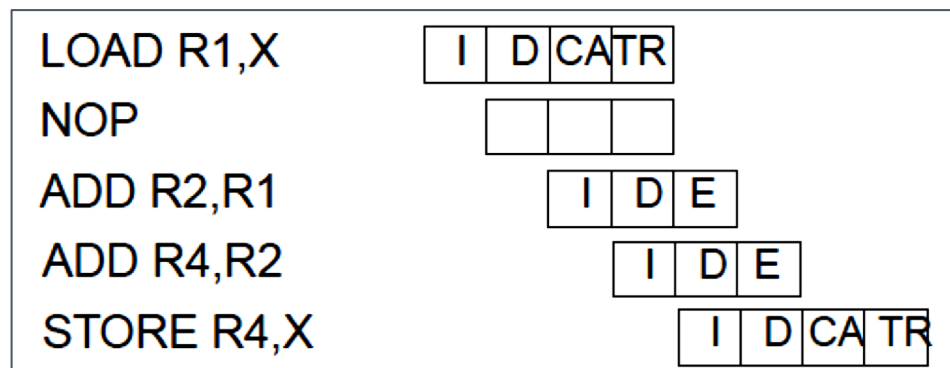
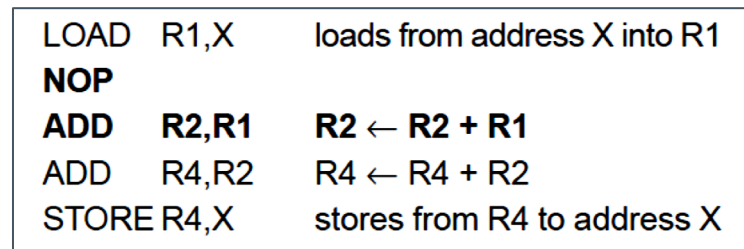
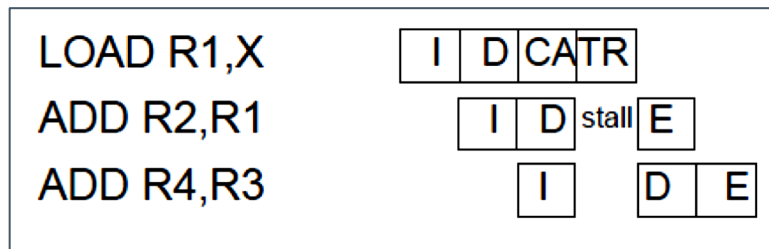
b)

CA: compute address, TR: transfer

Optimization of Pipelining

› **DELAYED LOAD**

– Đối với chuỗi sau, trình biên dịch đã tạo NOP sau LOAD vì không có lệnh nào để lấp đầy khe trống tải:



Optimization of Pipelining

› **LOOP UNROLLING**

- Sao chép phần thân của một vòng lặp một số lần được gọi là hệ số unrolling (u) và lặp lại theo bước u thay vì bước 1.
- Unrolling có thể cải thiện hiệu suất bằng cách
 - › Giảm chi phí vòng lặp
 - › Tăng tính song song của lệnh bằng cách cải thiện hiệu suất đường ống
 - › ..

Optimization of Pipelining

› *LOOP UNROLLING*

```
do i=2, n-1
    a[i] = a[i] + a[i-1] * a[i+1]
end do
```

(a) Original loop

```
do i=2, n-2, 2
    a[i] = a[i] + a[i-1] * a[i+1]
    a[i+1] = a[i+1] + a[i] * a[i+2]
end do

if (mod(n-2, 2) = 1) then
    a[n-1] = a[n-1] + a[n-2] * a[n]
end if
```

(b) Loop unrolled twice

- Chi phí vòng lặp bị cắt làm đôi vì hai lần lặp được thực hiện trước khi kiểm tra và phân nhánh nhánh ở cuối vòng lặp.
- Tính song song của các lệnh được tăng lên vì phép gán thứ hai có thể được thực hiện trong khi kết quả của lần đầu tiên đang được lưu trữ và các biến vòng lặp đang được cập nhật.
- Khi các phần tử mảng được gán cho các thanh ghi, $a[i]$ và $a[i + 1]$ được sử dụng hai lần trong thân vòng lặp, giảm số lượng tải mỗi lần lặp từ ba xuống còn hai.

CISC và RISC

- › CISC (Complex Instruction Set Computer)
 - Máy tính với tập lệnh phức tạp: mỗi lệnh có thể thực hiện càng nhiều chức năng càng tốt.
 - Hầu hết các máy tính để bàn hoặc máy tính xách tay sử dụng kiến trúc CISC (tập lệnh phức tạp) được sản xuất bởi Intel hoặc AMD.
 - Điện thoại thông minh và máy tính bảng sử dụng RISC (giảm tập lệnh tính toán) với ARM.

CISC và RISC

- › Các thiết kế của RISC có thể kế thừa một số tính năng CISC và rằng
- › Các thiết kế CISC có thể kế thừa một số tính năng của RISC.

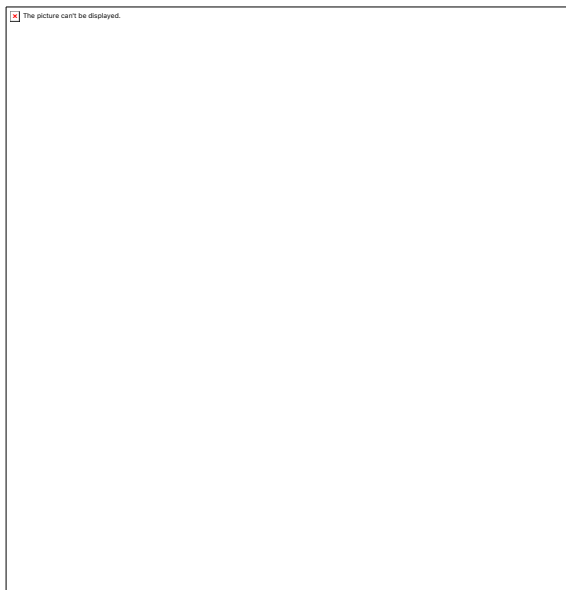
CISC và RISC

- › Các thiết kế RISC gần đây, đặc biệt là PowerPC, không còn là RISC thuần túy và các thiết kế CISC gần đây, đặc biệt là các mẫu Pentium II và Pentium sau này, kết hợp một số đặc điểm của RISC.
- › CISC được phát triển để làm cho trình biên dịch phát triển đơn giản hơn. Nó chuyển hầu hết gánh nặng của việc khởi tạo lệnh máy cho bộ xử lý.
 - Thay vì phải tạo một trình biên dịch viết các lệnh máy dài để tính căn bậc hai, bộ xử lý CISC sẽ có khả năng tích hợp sẵn để thực hiện việc này.

CISC và RISC

› Kiến trúc CISC

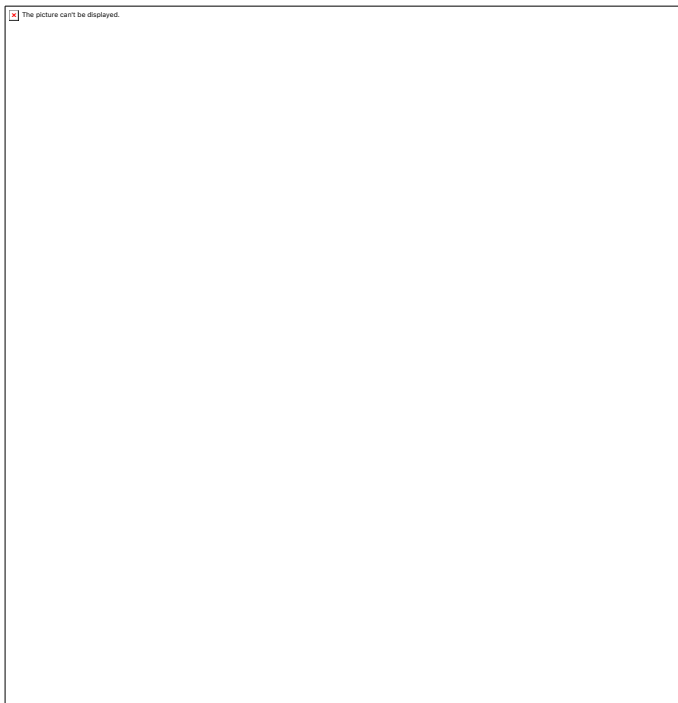
- Trực tiếp sử dụng bộ nhớ, thay vì tệp thanh ghi.
- Kiến trúc của CISC với bộ điều khiển được lập trình vi mô và bộ nhớ đệm.
- Kiến trúc này sử dụng bộ nhớ đệm để chứa cả dữ liệu và lệnh, do đó chúng chia sẻ cùng một đường dẫn cho cả lệnh và dữ liệu.



CISC và RISC

› Kiến trúc CISC

- CISC có các lệnh với định dạng chiều dài thay đổi. Do đó, số chu kỳ đồng hồ cần thiết để thực hiện các lệnh có thể thay đổi.
- Các lệnh trong CISC được thực hiện bởi chương trình vi mô có chuỗi vi lệnh.



CISC và RISC

› Ưu điểm của CISC (1)

- Lập trình vi mô dễ thực hiện và ít tốn kém hơn nhiều so với gắn cứng một bộ điều khiển.
- Dễ dàng để thêm các lệnh mới vào chip mà không thay đổi cấu trúc của tập lệnh vì kiến trúc sử dụng phần cứng đa năng để thực hiện các lệnh.

CISC và RISC

› Ưu điểm của CISC (2)

- Sử dụng hiệu quả bộ nhớ chính vì độ phức tạp (hoặc nhiều khả năng hơn) của lệnh cho phép sử dụng ít số lệnh hơn để đạt được một tác vụ nhất định.
- Trình biên dịch không cần quá phức tạp, vì tập lệnh chương trình vi mô có thể được viết để phù hợp với cấu trúc của các ngôn ngữ cấp cao.

CISC và RISC

› Nhược điểm của CISC

- Một phiên bản mới của bộ xử lý CISC bao hàm các bộ xử lý thế trước đó trong các tập hợp con của chúng. Do đó, phần cứng và tập lệnh của chip trở nên phức tạp với mỗi thế hệ bộ xử lý.
- Hiệu năng tổng thể của máy bị giảm do tốc độ xung nhịp chậm hơn.
- Sự phức tạp của phần cứng và phần mềm trên chip có trong thiết kế CISC để thực hiện nhiều chức năng.

CISC và RISC

› Ưu điểm của RISC (1)

- Hiệu suất của bộ xử lý RISC thường ***gấp hai đến bốn lần*** so với bộ xử lý CISC do tập lệnh được đơn giản hóa.
- Kiến trúc này sử dụng ít không gian chip hơn do tập lệnh giảm. Điều này cho phép đặt các chức năng bổ sung như các đơn vị số học dấu phẩy động, đơn vị quản lý bộ nhớ trên cùng một chip.

CISC và RISC

› Ưu điểm của RISC

- Chi phí cho mỗi chip được giảm bởi kiến trúc này sử dụng các chip nhỏ hơn bao gồm nhiều thành phần hơn trên một ***wafer silicon***.
- Bộ xử lý RISC có thể được thiết kế nhanh hơn bộ xử lý CISC do kiến trúc đơn giản của nó.
- Việc thực hiện các hướng dẫn trong bộ xử lý RISC cao do sử dụng nhiều thanh ghi để lưu trữ và thực thi so với bộ xử lý CISC.

CISC và RISC

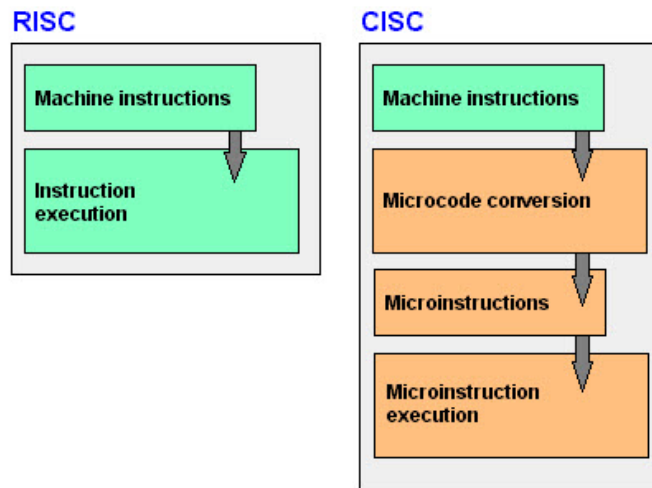
› Nhược điểm của RISC:

- Hiệu năng của bộ xử lý RISC phụ thuộc vào mã đang được thực thi.
- Bộ xử lý dành nhiều thời gian chờ đợi kết quả lệnh đầu tiên trước khi nó thực thi lệnh tiếp theo.
- Bộ xử lý RISC yêu cầu hệ thống bộ nhớ rất nhanh việc cung cấp các lệnh khác nhau. Thông thường, bộ nhớ cache lớn được cung cấp trên chip trong hầu hết các hệ thống dựa trên RISC.

CISC và RISC

› CISC nhanh hơn RISC?

- Bộ xử lý hiện đại có khá nhiều RISC. Ngay cả các bộ lệnh CISC (x86-64) được dịch sang mã vi mô RISC trên chip trước khi thực hiện. ...
- CPU RISC thường chạy ở tốc độ xung nhịp nhanh hơn CISC vì thời gian xung nhịp tối đa được quyết định bởi bước chậm nhất của đường ống (các hướng dẫn phức tạp hơn chậm hơn)



RISC và CISC

CISC	RISC
Kiến trúc CISC quan trọng hơn đối với phần cứng	Kiến trúc CISC quan trọng hơn đối với phần mềm
Tập lệnh phức tạp	Tập lệnh được giản lược
Truy cập bộ nhớ trực tiếp	Các lệnh khác thực thi chỉ với các thanh ghi
Lập trình rất đơn giản , ngắn gọn.	Lập trình cần nhiều dòng lệnh
Các lệnh phức tạp, cần nhiều chu kỳ để thực hiện	Các Lệnh đơn giản, cần chu kỳ nằm trong trình biên dịch duy nhất để thực hiện
Sự phức tạp nằm trong chương trình vi mô	Sự phức tạp nằm trong trình biên dịch
Độ dài các lệnh đa dạng	Độ dài các lệnh cố định
.....	

HẾT CHƯƠNG 8